

Linux Essentials

ICCS121 Syskills | MUIC

Lecture 02 | Filesystem, commands, permissions, and pipes

Who Am I?

- ผศ.ดร. ทรงพล ตีระกนก (Asst. Prof. Dr. Songpon Teerakanok)
- Assistant Professor, MUIC — Computer Science
- Founder, CYBERSKILLS Professional Training Center
- Research: Cybersecurity, Network Security, AI Security
- Email: songpon.tee@mahidol.ac.th

Disclaimer

- Slides are a **summary** — not a complete reference
- Always verify commands in your own terminal
- Some behavior may differ by OS version or distro
- For authoritative info: `man` pages, official docs

Agenda

- 1.** The Linux Filesystem
- 2.** Navigating the Filesystem
- 3.** File and Directory Operations
- 4.** Wildcards and Globbing
- 5.** Manual Pages
- 6.** Redirection and Pipes
- 7.** RC Files
- 8.** Users, Groups, and Permissions
- 9.** PATH Variable + Process Management + Useful Tools

The Linux Filesystem

Everything is a file

Everything Is a File

- Linux exposes **everything** through the file abstraction
- Regular files, directories, devices, sockets — all use the same interface
- Same tools (`cat` , `echo` , `>`) work on all file types

```
cat /proc/cpuinfo      # CPU info – read like a text file
cat /etc/hostname      # system hostname – also a file
echo "test" > /dev/null # write to the "black hole"
```

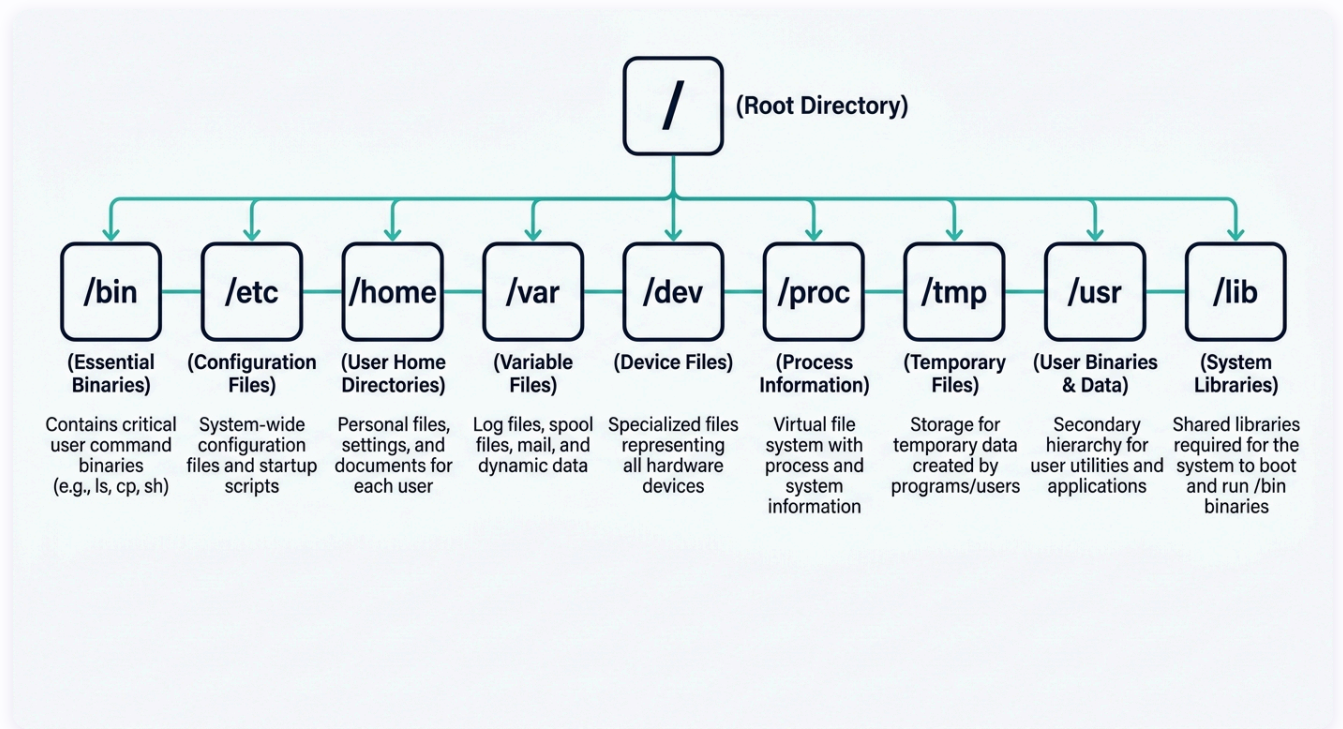
File Types

Symbol	Type	Example
-	Regular file	text, binary, image, executable
d	Directory	/home/student/
l	Symbolic link	shortcut to another file
b	Block device	/dev/sda (hard disk)
c	Character device	/dev/tty (terminal)
p	Named pipe (FIFO)	inter-process communication
s	Socket	network/local IPC endpoint

The Directory Tree

```

/
├── bin/    Essential user commands (ls, cp, cat, bash)
├── sbin/   System admin commands (fdisk, iptables)
├── etc/    Configuration files (passwd, hosts, ssh/)
├── home/   User home directories (/home/student/)
├── tmp/    Temporary files (cleared on reboot)
├── usr/    User programs and shared libraries
├── var/    Variable data – logs (/var/log/), databases
├── dev/    Device files (sda, null, urandom, tty)
├── proc/   Virtual FS – live kernel and process info
└── lib/    Shared libraries (like .dll on Windows)
    
```



src: ai-generated

Navigating the Filesystem

Absolute and relative paths

Absolute vs. Relative Paths

Absolute Path

- Starts with `/`
- Full path from root
- Works from anywhere

```
cd /home/student/hw1
```

Relative Path

- Starts from current directory
- Uses `.` (here) and `..` (parent)
- Shorter but context-dependent

```
cd ../projects/hw1
```

Navigation Commands

```
Terminal showing pwd, ls -la, and cd commands in action

student@crucible:~$ pwd
/home/student

student@crucible:~$ ls -la ~
total 48
drwx-----@ 12 student  staff   384 Apr 19 14:15 .
drwx-----@ 409 student  staff 13088 Apr 19 14:15 ..
-rw-r--r--@  1 student  staff   15 Apr 19 14:15 .bash_logout
-rw-r--r--@  1 student  staff   10 Apr 19 14:15 .bashrc
-rw-r--r--@  1 student  staff   11 Apr 19 14:15 .profile
drwxr-xr-x@  2 student  staff   64 Apr 19 14:15 Desktop
drwxr-xr-x@  2 student  staff   64 Apr 19 14:15 Documents
drwxr-xr-x@  2 student  staff   64 Apr 19 14:15 Downloads
drwxr-xr-x@  2 student  staff   64 Apr 19 14:15 lab
-rw-r--r--@  1 student  staff    8 Apr 19 14:15 lab1.py
-rw-r--r--@  1 student  staff   12 Apr 19 14:15 notes.txt
-rw-r--r--@  1 student  staff   20 Apr 19 14:15 README.md

student@crucible:~$ cd /etc && pwd
/etc
```

Command	Purpose	Example
pwd	Print working directory	pwd → /home/student
cd	Change directory	cd /etc
ls	List directory contents	ls -la
ls -l	Long format (permissions, size)	ls -l /home
ls -a	Show hidden files (. prefix)	ls -a

Reading `ls -l` Output

```
drwxr-xr-x  5 student student 4096 Jan 12 14:30 projects
```

Diagram illustrating the components of the `ls -l` output line:

- Permissions: `drwxr-xr-x`
- Hard link count: `5`
- File owner: `student`
- Group owner: `student`
- Size (bytes): `4096`
- Date and Time: `Jan 12 14:30`
- Name: `projects`

Legend for file type and permissions:

- `d`: directory
- `-`: file
- `l`: link
- `r`: read permission
- `w`: write permission
- `x`: execute permission

File and Directory Operations

Create, remove, move, copy, view

Creating Files and Directories

```
touch newfile.txt      # create empty file (or update timestamp)
mkdir mydir            # create directory
mkdir -p a/b/c         # create nested directories (parents too)
```

- `touch` has dual purpose: create **or** update timestamp
- `mkdir -p` creates parent directories automatically

Removing Files and Directories

```
rm file.txt           # remove a file (no confirmation!)
rm -i file.txt        # remove with confirmation prompt
rm -r mydir/          # remove directory and all contents
rm -rf mydir/         # force remove – no prompts – DANGEROUS
rmdir emptydir/       # remove ONLY if empty
```

WARNING

`rm` does NOT move files to a trash can. Deleted files are **gone permanently**. There is no undo. Always double-check before using `rm -rf .`

Moving and Copying

```
cp source.txt dest.txt           # copy file
cp -r srcdir/ destdir/           # copy directory recursively
mv oldname.txt newname.txt       # rename file
mv file.txt /home/student/       # move to different directory
```

- `mv` serves dual purpose: **rename** and **move**
- `cp -r` is required for directories (recursive copy)

Viewing File Contents

Command	Best For	Example
cat	Short files	cat config.txt
less	Long files (scrollable)	less /var/log/syslog
head -n 10	Quick preview (first lines)	head -n 20 data.csv
tail -n 10	Log files (last lines)	tail -n 10 auth.log
tail -f	Monitoring live logs	tail -f /var/log/auth.log

Wildcards and Globbing

Pattern matching for files

Wildcard Characters

Wildcard	Meaning	Example
*	Zero or more characters	<code>ls *.c</code> — all C files
?	Exactly one character	<code>ls file?.txt</code> — file1.txt, not file10.txt
[...]	One char from the set	<code>ls [0-9]*</code> — starts with a digit
[! ...]	One char NOT in the set	<code>ls [!A-Z]*</code> — not starting with uppercase

```
ls *.c           # all C source files
file /home/user/* # check type of everything in a directory
mv *.jpg images/  # move all JPEGs to images/
```

Manual Pages

The built-in documentation system

Using `man`

```
man grep           # open grep manual
man 3 printf        # C library printf (section 3)
```

Section	Contents	Example
1	User commands	<code>man 1 ls</code>
2	System calls	<code>man 2 open</code>
3	Library functions	<code>man 3 printf</code>
5	File formats	<code>man 5 passwd</code>
8	Admin commands	<code>man 8 mount</code>

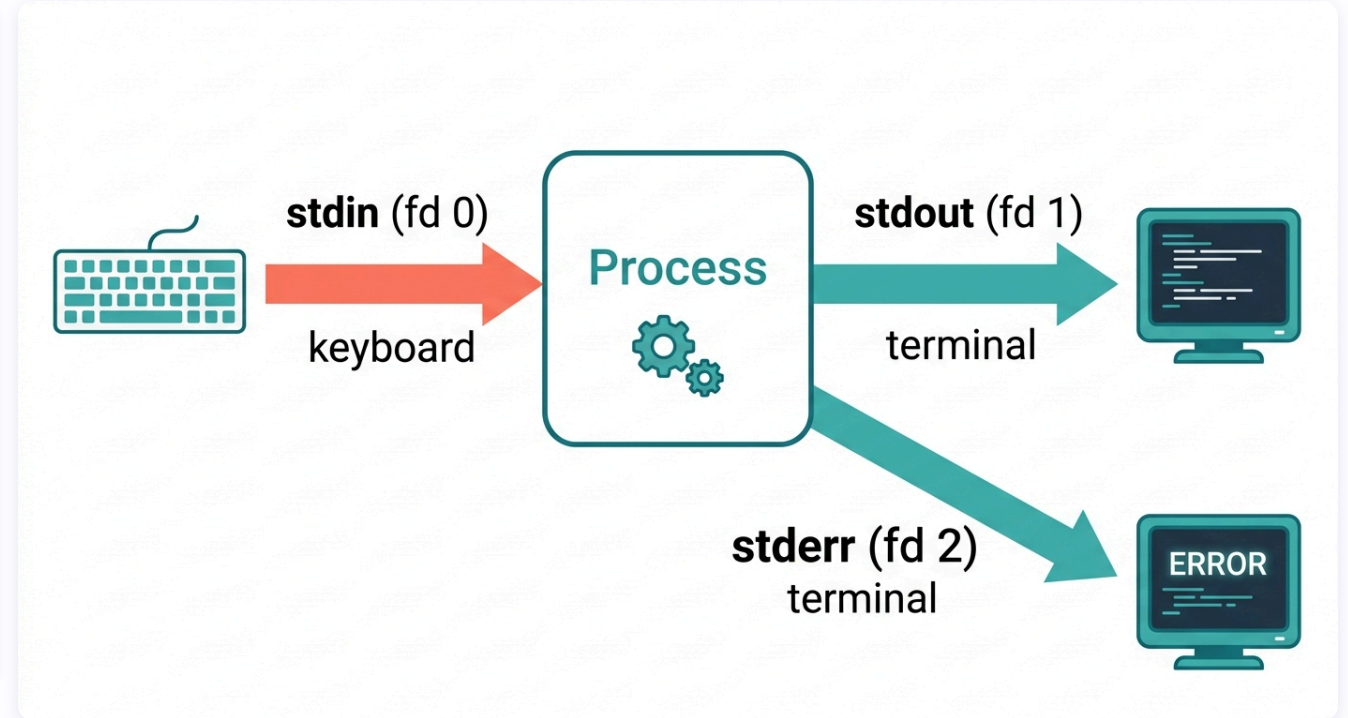
- Navigation: arrows to scroll, `/pattern` to search, `n` next, `q` quit

Redirection and Pipes

Composing commands — the UNIX way

Standard Streams

Stream	File Descriptor	Default	Purpose
stdin	0	Keyboard	Input to the program
stdout	1	Terminal	Normal output
stderr	2	Terminal	Error messages



src: ai-generated

- Every process gets these three streams automatically
- Redirection changes **where** they point

Output Redirection

```
ls > myfiles.txt           # stdout → file (OVERWRITE)
ls >> myfiles.txt          # stdout → file (APPEND)
command 2> errors.txt      # stderr → file
command &> all.txt         # both stdout + stderr → file
```

- **>** **overwrites** the file (creates if missing)
- **>>** **appends** to the file (creates if missing)

Input Redirection and Pipes

Input Redirection

```
sort < unsorted.txt  
wc -l < data.csv
```

Feed file contents as stdin

Pipes

```
ls | more  
ls -la | grep config  
cat file | wc
```

Connect stdout → stdin

Pipeline Example

STEP 1



src: ai-generated

```
# Find top 5 largest files in /var/log
du -sh /var/log/* | sort -rh | head -5
```

- `du -sh` — disk usage, summary, human-readable
- `sort -rh` — sort reverse, human-readable numbers
- `head -5` — first 5 lines only
- Each `|` passes output to the next command

RC Files

Customizing your shell environment

Shell Startup Files

```
# Common .bashrc additions  
alias ll='ls -la'  
alias gs='git status'  
export EDITOR=vim  
export PATH="$HOME/bin:$PATH"
```

- `~/.bashrc` — runs every new bash shell (interactive)
- `~/.zshrc` — equivalent for zsh (macOS default)
- `~/.bash_profile` — runs for login shells only
- After editing: `source ~/.bashrc` to apply immediately

Users, Groups, and Permissions

Who can do what to which files

Users and Groups

```
whoami          # show current username
id              # show UID, GID, and all groups
groups          # list groups you belong to
```

- Every user has a **username**, **UID**, **home directory**
- Users can belong to **multiple groups**
- Groups enable shared access to files

Superuser and Root

Root (UID 0)

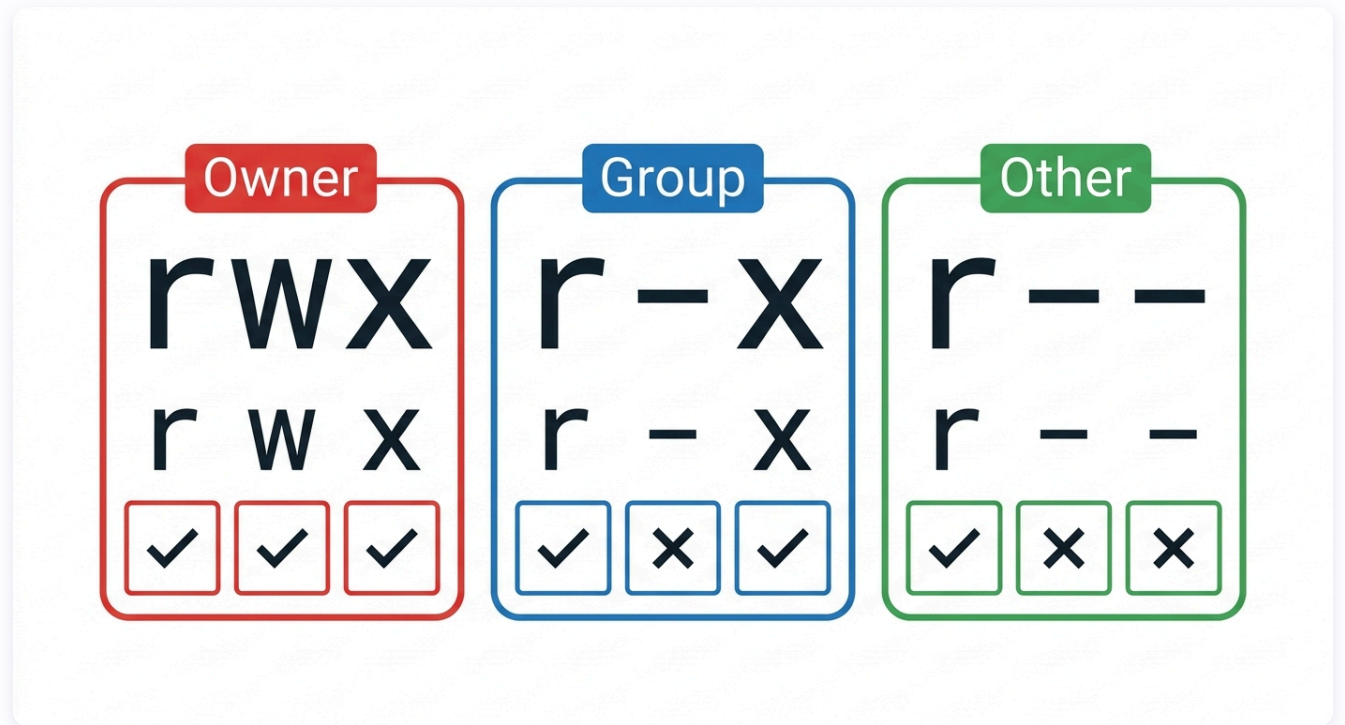
- Most privileged user
- Can override **all** permissions
- Access any file, kill any process
- Modify any system setting

Two ways to use root:

```
# su - switch to root user  
su # requires root password  
  
# sudo - run ONE command as root  
sudo apt update # your password  
sudo cat /etc/shadow
```

File Permissions — Three Sets

Set	Applies To	Meaning
Owner (u)	File creator/owner	What the owner can do
Group (g)	Group members	What group members can do
Other (o)	Everyone else	What all others can do



src: ai-generated

What Each Permission Bit Does

Permission	On File	On Directory
Read (r)	View contents	List contents (<code>ls</code>)
Write (w)	Modify contents	Create/delete files
Execute (x)	Run as program	Enter directory (<code>cd</code>)

Reading Permission Strings

```
drwxr-xr-x  5 joe student 4096 May 12 2015 joe
|  |  |  |
|  |  |  └─ other: r-x (read + execute, no write)
|  |  └─ group: r-x (read + execute, no write)
|  └─ owner: rwx (read + write + execute)
└─ type: d = directory
```

Octal notation: `rwxr-xr-x` = **755**

- Owner: `rwx` = $4+2+1 = 7$
- Group: `r-x` = $4+0+1 = 5$
- Other: `r-x` = $4+0+1 = 5$

Changing Permissions — `chmod`

Octal Mode

```
chmod 755 script.sh  
chmod 644 document.txt  
chmod 700 private/
```

Symbolic Mode

```
chmod u+x script.sh  
chmod g-w file.txt  
chmod o-rwx private.txt  
chmod a+r readme.txt
```

- `u` = owner, `g` = group, `o` = other, `a` = all
- `+` = add, `-` = remove, `=` = set exactly

Changing Ownership — **chown**

```
sudo chown alice file.txt           # change owner
sudo chown alice:developers file.txt # change owner AND group
sudo chown :developers file.txt      # change group only
sudo chown -R alice:devs project/    # recursive – entire tree
```

- Only **root** (or **sudo**) can change file ownership
- **-R** applies changes recursively to all subdirectories

PATH Variable

How the shell finds commands

How the Shell Finds Commands

```
echo $PATH  
# /usr/local/bin:/usr/bin:/bin:/usr/sbin:/sbin
```

When you type `ls`:

1. Shell checks `/usr/local/bin/ls` — not found
2. Shell checks `/usr/bin/ls` — not found
3. Shell checks `/bin/ls` — **found!** Runs it
4. If not found anywhere → `command not found`

```
which ls           # shows: /bin/ls  
which python3      # shows where python3 lives
```

Process Management Preview

Managing running programs

Viewing Processes

```
ps           # processes in current terminal
ps aux       # ALL processes on the system
pstree       # parent-child tree visualization
top          # live process monitor (q to quit)
```

- Every process has a **PID** (unique number) and **PPID** (parent)
- Your terminal runs a shell → shell is parent of commands you run

Controlling Processes

Command	Description
<code>kill <PID></code>	SIGTERM — graceful termination
<code>kill -9 <PID></code>	SIGKILL — force kill (cannot be ignored)
<code>Ctrl+C</code>	SIGINT — interrupt current program
<code>Ctrl+Z</code>	Suspend (pause) foreground process
<code>jobs</code>	List suspended/background jobs
<code>fg</code>	Resume in foreground
<code>bg</code>	Resume in background

Useful Tools & Terminal Tips

Productivity essentials

Download and Archive Tools

Command	Purpose	Example
wget	Download a file	wget https://example.com/data.csv
curl	HTTP client (flexible)	curl -O https://example.com/file
tar -czf	Create gzipped archive	tar -czf backup.tar.gz dir/
tar -xzf	Extract gzipped archive	tar -xzf archive.tar.gz
zip / unzip	.zip files	zip archive.zip file1 file2

Essential Keyboard Shortcuts

Shortcut	Action
Tab	Autocomplete file/folder names
Ctrl+C	Kill the current program
Ctrl+R	Search command history
Ctrl+L	Clear the screen
Ctrl+D	Send EOF — exits the shell
Ctrl+A / Ctrl+E	Jump to start / end of line
!!	Repeat the last command
sudo !!	Rerun last command with sudo

Key Takeaways

1. **Everything is a file** — devices, pipes, sockets share the file interface
2. **Absolute paths** start with `/`; **relative paths** use `.` and `..`
3. `rm` is **permanent** — no trash can, no undo
4. **Pipes** (`|`) chain stdout → stdin — the UNIX superpower
5. **Permissions**: rwx × owner/group/other; octal 755, 644
6. **PATH** tells the shell where to find commands
7. **Tab** and **Ctrl+R** are your best friends in the terminal

Thank You

ICCS121 L02: Linux Essentials



Q&A

All questions are welcome



EMAIL

songpon.tee@mahidol.ac.th



OFFICE HOURS

By appointment · MUIC